QueryAPIGate HOW-TO GUIDE 46 · AGENTS

Let AI clients sign in through your identity provider

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=46-let-ai-clients-sign-in-through-your-identity-provider

Time: 30 minutes. **You'll end up with:** an AI client - a connector in a hosted assistant, or your own agent - connecting to your MCP server by signing its user in through your identity provider, not with an API key you hand out: the user's role decides what the agent can reach, its masking rules apply, every run is recorded under the user's name, and a token issued for anything else is refused.

> **Stable** - OAuth for the MCP server is under the compatibility promise since 0.30 (what that means).

Every command and response below was run against a real MCP server and a real Keycloak 26 (in Docker), with the example APIs loaded, and a real MCP client - the Python SDK's - going through the whole sign-in: discovery, the login page, the authorization code with PKCE, the token, the tool call.

How it fits together

```text
AI client --(1) POST /mcp, no token⟶  MCP server ⟶ 401 + where to sign in
          --(2) reads /.well-known/oauth-protected-resource ⟶ "tokens come from <your provider>"
          --(3) signs the user in at your provider, gets a token for the MCP server
          --(4) POST /mcp, Authorization: Bearer <token> ⟶ tools, as the user's role
```

QueryAPIGate only **checks** tokens. Your identity provider issues them, signs the user in (with your MFA, your policies), and registers the client. QueryAPIGate never sees a password and issues no tokens of its own.

Step 1: At your provider - a scope whose tokens name the MCP server

A token is accepted only if its audience (`aud`) is the MCP server's own URL - so a token your provider issues for some other application can't be used here. Make that happen with a scope. In Keycloak, in your realm:

1. **Client scopes** → **Create client scope**: name `queryapigate.mcp`, type *Optional*, *Include in token scope* on.
2. In it, **Mappers** → **By configuration** → **Audience**: *Included Custom Audience* `https://data.example.com/mcp` (the MCP server's URL, exactly as clients reach it), *Add to access token* on.

A token requested with `scope=queryapigate.mcp` now carries:

```json
{"iss": "http://localhost:58080/realms/qag-test", "aud": ["http://127.0.0.1:5001/mcp", "account"],
 "sub": "7222e50f-b640-4e00-ab30-da4c0c9c9261", "azp": "agent", "scope": "openid queryapigate.mcp
profile email",
 "exp": 1791692237, ...}
```

(In this run the MCP server's URL was `http://127.0.0.1:5001/mcp`; yours is its public `https://` URL.)

Other providers have the same idea under another name - an *API*, *resource* or *application ID URI* whose identifier becomes the token's audience, and a scope on it. Whatever yours calls it, check a real token: `aud` must be the MCP server's URL, `scope` (or `scp`) must hold `queryapigate.mcp`, and `sub` must be there.

Step 2: In QueryAPIGate - a role for agents

What a signed-in agent may do is a role's, as for any signed-in user:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/roles -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "agents", "connections": ["examples"],
  "mask": [{"column": "customer.email", "action": "hash"}]
}'
```

Give it as little as the agents need: saved queries only (a `queries` or `collections` grant, no `connections`) is the safest; a connection grant adds the ad-hoc `execute_sql` and `list_tables` tools, and masking hides what they shouldn't see.

Step 3: Turn it on

```shell
QUERYAPIGATE_MCP_RESOURCE_URL=https://data.example.com/mcp \
QUERYAPIGATE_MCP_AUTH_SERVER=https://idp.example.com/realms/acme \
QUERYAPIGATE_MCP_ROLE=agents \
queryapigate mcp
```

Here, against the local Keycloak:

```text
MCP sign-in through http://localhost:58080/realms/qag-test, for tokens issued to
http://127.0.0.1:5001/mcp (scope queryapigate.mcp)
INFO:     Uvicorn running on http://127.0.0.1:5001 (Press CTRL+C to quit)
```

- `QUERYAPIGATE_MCP_AUTH_SERVER` is the provider's **issuer** - for Keycloak, `https://<host>/realms/<realm>`. Its keys are found from its published metadata; if that can't be fetched, the log says so at startup and every token is refused until it can.
- `QUERYAPIGATE_MCP_ROLE_CLAIM` instead (or as well) lets the token pick the role: a claim holding a role name, or a list - the first that exists wins.
- Both URLs must be `https://` (only `localhost` may be plain HTTP), and a hosted client reaches the MCP server from the internet: put it behind a reverse proxy with real TLS.

Step 4: Check it the way a client will

The metadata a client reads first:

```shell
curl https://data.example.com/.well-known/oauth-protected-resource/mcp
```

```json
{"resource": "http://127.0.0.1:5001/mcp", "authorization_servers": ["http://localhost:58080/realms/qag-test"],
 "scopes_supported": ["queryapigate.mcp"], "bearer_methods_supported": ["header"], "resource_name": "QueryAPIGate"}
```

And the answer to a request with no token - the client's cue to sign the user in:

```shell
curl -i -X POST https://data.example.com/mcp/ -H 'Content-Type: application/json' \
  -H 'Accept: application/json, text/event-stream' -d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
```

```text
HTTP/1.1 401 Unauthorized
www-authenticate: Bearer resource_metadata="http://127.0.0.1:5001/.well-known/oauth-protected-resource/mcp", scope="queryapigate.mcp"

{"error": "Unauthorized: missing or invalid X-API-Key or bearer token", "code": "unauthorized", "request_id": "..."}
```

Step 5: Register the client

The client needs a client ID at your provider. There are three ways, and which one works is between the client and your provider - QueryAPIGate plays no part:

- **Register it yourself** (any provider): create a client - in Keycloak **Clients** → **Create client**, *Client authentication* off (a public client, using PKCE), *Standard flow* on, the client's redirect URL under *Valid redirect URIs* - then add `queryapigate.mcp` under its **Client scopes** as *Optional*, and give the client ID to whoever configures the AI client.

  Creating clients through Keycloak's admin API or a realm import? Create the client, then add the scope: a client created with its scopes listed gets none of Keycloak's default ones - `basic`, which puts `sub` in the token, among them - and its tokens are then refused (*the access token names no user*).

- **Client ID Metadata Documents**: the client's ID is an HTTPS URL describing it, which the provider fetches - nothing to register, if your provider supports it.

- **Dynamic Client Registration**: the client registers itself. Keycloak supports it, but two of its default policies refuse an anonymous registration, as this run found in turn: *Allowed Client Scopes* (a client asking for `queryapigate.mcp`, until the scope was made a realm-wide optional one), then *Trusted Hosts* (a request from a host not on its list). Opening that up lets anyone register a client in your realm: decide it deliberately, or register clients yourself.

Step 6: Sign in, and call a tool

With the Python SDK's OAuth client - the same steps a hosted client takes, the login page in a browser:

```python
from mcp.client.auth import OAuthClientProvider
from mcp.shared.auth import OAuthClientMetadata

auth = OAuthClientProvider(
    "https://data.example.com/mcp",
    OAuthClientMetadata(client_name="My agent", redirect_uris=["http://127.0.0.1/callback"],
                        grant_types=["authorization_code", "refresh_token"], response_types=["code"],
                        token_endpoint_auth_method="none"),
    storage, redirect_handler=open_browser, callback_handler=wait_for_callback)

async with streamablehttp_client("https://data.example.com/mcp", auth=auth) as (read, write, _):
    async with ClientSession(read, write) as session:
        await session.initialize()
        await session.call_tool("execute_sql", {"connection_name": "examples",
                                                "sql": "SELECT first_name, email FROM customer LIMIT
2"})
```

The client got the 401, read the metadata, found Keycloak's, and sent the user to sign in with `scope=queryapigate.mcp`, a PKCE challenge, and `resource=http://127.0.0.1:5001/mcp`. After signing in:

```text
Tools: 13 ['example_all_rentals', 'example_film_lookup', 'example_film_search'] ...
Result: {'rows': [{'first_name': 'Isla', 'email': 'ae33faddd7f39b68'},
                  {'first_name': 'Ava', 'email': '3cdc177ae6b31f79'}], 'truncated': False}
```

The role's masking rule hashed the email. The run is in history under the user:

```json
{"key_name": "jwt:7222e50f-b640-4e00-ab30-da4c0c9c9261", "transport": "mcp", "status": "success",
"rows": 2,
 "masked": [{"column": "email", "action": "hash", "source": "customer.email"}], ...}
```

So `GET /api/v1/history?key=jwt:<sub>` is everything this person's agents did.

What's refused, and how

THE TOKEN	ANSWER	SO THE CLIENT...
none, or not a token	`401`, `resource_metadata` and `scope`	signs the user in
issued for another audience (your REST API, another app), another issuer, expired, unknown key	`401` `error="invalid_token"`	signs the user in again
valid, without `queryapigate.mcp`	`403` `insufficient_scope`	asks for the scope
valid, but its user has no role here, or the role's `allowed_ips` leave the address out	`403` `forbidden`	stops - signing in again wouldn't help

Tokens are kept apart. A token for your REST API is refused on `/mcp` however valid, and REST refuses a token naming the MCP server - one leaked from either can't be replayed against the other. API keys keep working on `/mcp`, for scripts and SDKs.

Good to know

- **Revoking access.** A token can't be recalled before it expires - keep access tokens short-lived (minutes, as providers do by default; the client refreshes them). Disabling the user at your provider stops new tokens; disabling or narrowing the role here takes effect on the next request.
- **The resource URL is an identity, not a location**: clients compare it with the URL they connected to, and the provider puts it in `aud`. Set it to the public URL, `https://data.example.com/mcp` - not the internal address the server listens on.
- **A scope named by URI** (`api://queryapigate/queryapigate.mcp`, as some providers require): set `QUERYAPIGATE_MCP_SCOPE` to it; a token carrying its last part, `queryapigate.mcp`, is accepted.
- **Which person was it?** The `sub` claim - for Keycloak, a user's ID. `from_claim` parameters bind to the token's claims, so a saved query can return only the signed-in user's rows.

Next steps

- Let an AI agent call your saved queries via MCP - the same server, with an API key.
- Hide personal data from a partner or an AI agent.
- MCP server reference: Sign-in for AI clients.